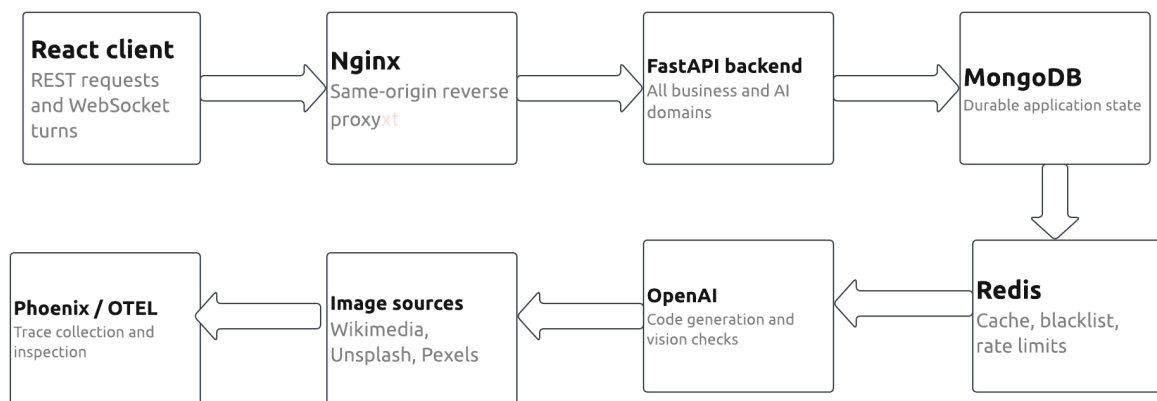


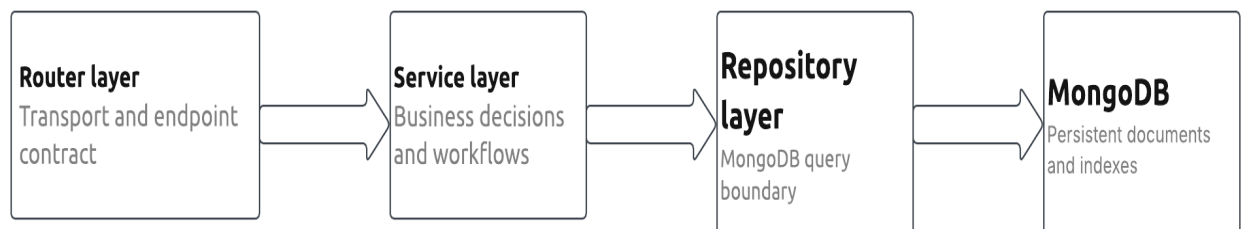
Backend overview

The backend is an asynchronous FastAPI modular monolith. One deployable application exposes versioned REST APIs and a streaming WebSocket endpoint, implements authentication and project/file/chat domains, orchestrates the AI engine, persists durable state in MongoDB, uses Redis for ephemeral state and caching, calls OpenAI for generation, and exports traces through OpenTelemetry.

System boundary



Layered architecture



Router layer

- Defines URL, HTTP method, status code, and response model.
- Receives Pydantic-validated request values.
- Resolves current user and services through dependencies.
- Delegates exactly one application use case to a service.
- Converts expected failures into the shared API envelope.

Service layer

- Enforces project ownership and workspace restrictions.
- Coordinates multiple repositories and Redis cache invalidation.
- Creates starter projects and manages cascade deletion.
- Orchestrates chat, undo, AI modifications, and repair.
- Provides the transaction-like application boundary.

Repository layer

- Encapsulates collection names and Motor query syntax.
- Converts external string IDs into Mongo ObjectIds.
- Provides reusable create/get/list/update/delete operations.
- Supports chat pagination, undo lookup, and file-path lookup.
- Makes services independent of low-level Mongo API details.

Schema and model layers

- Pydantic schemas are API DTOs: they validate input, describe output, serialize values, and generate OpenAPI documentation.
- Model modules construct Mongo documents with timestamps and internal fields. They are not ORM entities; repositories still own persistence.
- Keeping DTOs separate prevents raw database representation from becoming the public API contract.

Backend technology stack

Technology	Architectural purpose	Reason for selection
FastAPI	REST, dependency injection, WebSocket hosting	Async-first Python API framework with OpenAPI support
Uvicorn	ASGI runtime	Runs HTTP and long-lived WebSocket connections
Pydantic	Configuration and API DTO validation	Typed validation and serialization
Motor / PyMongo	Asynchronous MongoDB access	Document model fits dynamic files and chat metadata
Redis	Ephemeral security and cache state	Low-latency TTL keys and counters
OpenAI SDK	Streaming and non-streaming model calls	Async API and usage reporting
httpx	Image provider search and validation	Asynchronous outbound HTTP
OpenTelemetry	Distributed-style tracing	Measures AI and request pipeline stages
Phoenix	Trace inspection	AI-oriented observability UI
Docker Compose	Local/runtime topology	Repeatable multi-container environment
Nginx	SPA serving and reverse proxy	Single browser origin for /api and /ws

Domain services

Service	Primary responsibility	Dependencies
AuthService	Signup, login, refresh, logout, rate limits	User/refresh repos, Redis, JWT
ProjectService	Project lifecycle, ownership, starter files	Project/file/folder/chat repos, Redis
FileService	Tree and file/folder operations	File/folder repos, Redis
ChatService	History, AI turn delegation, undo	Chat repo, AIPipeline
WorkspaceService	Workspace catalog	Static domain metadata

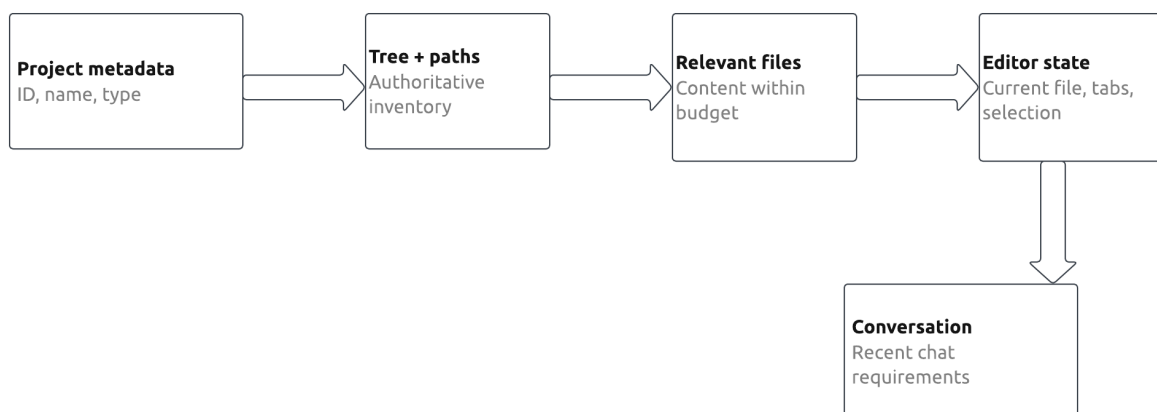
AI module architecture

The AI engine is an in-process bounded module, not a separate microservice. It isolates request classification, project context assembly, prompt policy, provider access, structured response parsing, file application, image resolution, guardrails, telemetry, and preview repair.

AI component map

Component	Input	Output / responsibility
RequestRouter	User request + workspace state	Category, model tier, temperature, token limit
ContextBuilder	Project/files/history/editor state	Budgeted ProjectContext
PromptBuilder	Context + system registry + assets	Ordered system/user LLM messages
OpenAIProvider	Messages and generation settings	Completion or streamed deltas
ResponseParser	Raw assistant content	Prose + structured file proposals
FileModifier	Validated proposals	Applied changes + reverse changes
Preview integrity	Current generated project	Import/entry/asset issues
Repair loop	Integrity issues + current project	Corrective file operations

Context Builder



AI turn lifecycle

Stage	Purpose	Input	Output
• Guard	Reject or sanitize unsafe input	User text	Safe request or blocked error
• Route	Choose specialized behavior and model tier	Request + workspace	RoutingDecision
• Context	Ground AI in actual project	Files/editor/history	ProjectContext
• Assets	Resolve required website imagery	Brief + prior requirements	Validated subject URL groups
• Prompt	Assemble policy and evidence	Context + route + assets	LLM messages
• Generation	Produce explanation and file operations	Messages	Streamed/full model output
• Parsing	Separate prose from mutations	Raw output	ParsedAIResponse
• Modification	Apply authorized changes	File proposals	Changes + reverse changes
• Verification	Detect preview blockers	Current project	Integrity issues
• Repair	Correct remaining blockers	Issues + project context	Additional applied changes

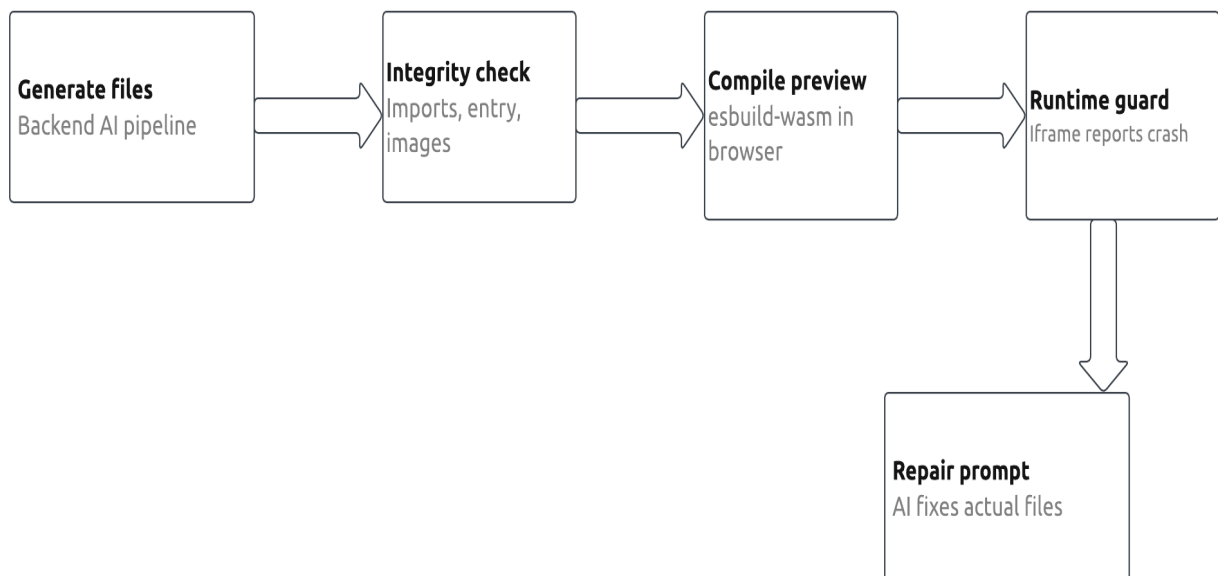
AI file modification pipeline

- The system prompt requires full file blocks with explicit actions.
- ResponseParser validates create/update/delete syntax.
- Workspace policy restricts editable file types where required.
- FileModifier delegates persistence to FileService.
- Reverse changes are stored with the assistant message.
- Undo replays reverse operations through normal services.

Website Builder backend architecture

- Website requests receive a specialized production design prompt.
- Asset resolution detects the domain and required image roles.
- OpenAI vision conservatively checks named-person identity.
- Only validated subject-labelled URLs enter the generation prompt.
- Backend integrity checks imports, React entry, and asset use.
- Frontend separately compiles and reports runtime failures.

Website preview flow across backend and frontend



Database access architecture

Collection	Ownership	Important indexes / relationships
users	Auth domain	Unique email
projects	Project domain	user_id + workspace_type; user_id + updated_at
files	File domain	Unique project_id + path; project_id + folder_id
folders	File domain	Unique project_id + path
chat_sessions	Chat domain	Unique project_id
chat_messages	Chat domain	session_id + created_at
refresh_tokens	Auth domain	Unique token_hash; TTL expires_at

Redis architecture

- `jwt:blacklist:*` stores revoked access tokens.
- `rate:login:*` limits repeated login attempts.
- `project:tree:*` caches serialized file trees.
- `recent:file:*` improves AI context relevance.
- `ai:context:*` stores short-lived context metadata.

External services

- OpenAI: generation, streaming, image requirement extraction.
- Wikimedia/Unsplash/Pexels: candidate visual assets.
- Phoenix: receives OTLP traces for AI inspection.
- Nginx: serves frontend and proxies backend transports.
- MongoDB and Redis: separately containerized infrastructure.

Normal REST flow

- Client → Nginx → FastAPI middleware → JWT dependency → versioned router → service → repository → MongoDB → schema serialization → response envelope.

Streaming AI flow

- Client WebSocket → token handshake → ownership check → context and asset preparation → OpenAI stream → deltas to client → parse/apply → preview repair → final done event with file changes.

Data flow classifications

Data class	Source	Destination	Lifetime
Identity	Signup/login	MongoDB + refresh cookie	Persistent/session
Project source	Editor/AI/import	MongoDB files collection	Persistent
Chat content	User + AI	MongoDB chat_messages	Persistent
AI prompt context	Project + editor + chat	OpenAI request	Per turn
AI stream	OpenAI	WebSocket client	Transient
Tree cache	Mongo repositories	Redis	Short TTL
Traces	Backend spans	Phoenix collector	Observability retention

Operations :

Authentication and authorization architecture

Authentication

- Passwords are hashed with bcrypt.
- Short-lived access JWTs authenticate API requests.
- Refresh tokens are stored hashed and delivered by HTTP-only cookie.
- Logout revokes refresh state and blacklists the access token in Redis.
- Login attempts are rate-limited with expiring Redis counters.

Authorization

- Project ownership is the primary access-control boundary.
- File, folder, chat, AI, and undo operations resolve through that project.
- Workspace policies restrict editable extensions.
- WebSocket turns repeat ownership checks inside the pipeline.
- Cross-project context and modifications are excluded by project ID.

WebSocket architecture

Phase	Client/server action	Result
Connect	Open <code>/ws/chat/{project_id}</code>	Long-lived channel
Authenticate	First frame carries access token	User identity established
Submit	Prompt, active file, open tabs, apply flag	Turn preparation begins
Stream	Server emits start and delta events	Progressive UI response
Finalize	Server parses and applies file blocks	Durable project changes
Repair	Server may emit repair progress	Preview blockers corrected
Complete	Server emits done with metadata and changes	Client refreshes state

Error handling architecture

Failure class	Current handling	Architectural intent
Request validation	FastAPI/Pydantic 422 handler	Reject invalid DTOs early
Authentication	Typed AppException and status	Do not enter service flow
Authorization	Ownership service raises	Prevent cross-project access
LLM auth/rate/provider	Normalized provider exceptions	Stable client messages
Stream failure	WebSocket error event	Keep protocol consistent
Missing imports/runtime	Integrity issue + repair loop	Self-heal generated projects
Image provider failure	Alternative provider/curated fallback	Avoid broken visuals
Telemetry failure	Circuit-breaking exporter	Observability must not break requests
Unexpected backend error	Global 500 envelope + server log	Avoid leaking internals

Deployment and Docker architecture

Container	Responsibility	Connectivity
frontend	Nginx SPA + reverse proxy	Host :3000 → container :80
backend	FastAPI REST, WebSocket, AI engine	Internal :8000
mongodb	Primary document database	Internal :27017; host published for development
redis	Cache and security state	Internal :6379
phoenix	Trace receiver and UI	Host :6006